

```

"""
Q-Score: Quasi-Identifier Detection and Risk Assessment with NLP Enhancement
=====
Detects quasi-identifiers using spaCy NER + pattern matching and calculates
re-identification risk based on k-anonymity principles.
"""

import os
import re
import json
import requests
import spacy
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from enum import Enum
from typing import Optional, List, Tuple
from functools import lru_cache
from datetime import datetime, date
from pathlib import Path
from dotenv import load_dotenv

load_dotenv()

# =====
# Data Classes and Enums
# =====

class QIType(Enum):
    """Types of quasi-identifiers we detect."""
    AGE = "age"
    DATE_OF_BIRTH = "date_of_birth"
    GENDER = "gender"
    LOCATION = "location"
    ZIP_CODE = "zip_code"
    DISEASE = "disease"
    OCCUPATION = "occupation"
    ETHNICITY = "ethnicity"
    EDUCATION = "education"

@dataclass
class QuasiIdentifier:
    """Represents a detected quasi-identifier."""
    qi_type: QIType
    raw_value: str
    normalized_value: str
    confidence: float
    start_pos: int
    end_pos: int
    detection_method: str = "pattern" # "pattern", "ner", or "hybrid"

    def __hash__(self):
        return hash((self.qi_type, self.normalized_value))

@dataclass
class QIFrequency:
    """Frequency/probability information for a QI value."""
    qi_type: QIType
    value: str
    probability: float
    population_count: Optional[int] = None
    source: str = "unknown"

@dataclass
class QScoreResult:
    """Q-Score analysis result."""
    q_score: float

```

```

    expected_k: float
    detected_qis: list
    frequencies: dict
    explanation: str
    original_text: str
    masked_text: str

# =====
# NLP-Enhanced Quasi-Identifier Extractor
# =====
class NLPQIEExtractor:
    """
    Extracts quasi-identifiers using a combination of:
    1. spaCy NER for entity recognition
    2. Pattern matching for structured data
    3. Custom classification for QI categorization
    """

    def __init__(self, model_name: str = "en_core_web_lg"):
        """Initialize with spaCy model."""
        try:
            self.nlp = spacy.load(model_name)
        except OSError:
            print(f"Downloading spaCy model {model_name}...")
            spacy.cli.download(model_name)
            self.nlp = spacy.load(model_name)

        # Add custom entity ruler for medical/occupation terms
        self._add_custom_patterns()

    def _add_custom_patterns(self):
        """Add custom entity patterns for QI detection."""
        ruler = self.nlp.add_pipe("entity_ruler", before="ner",
        config={"overwrite_ents": False})

        patterns = [
            # Diseases
            {"label": "DISEASE", "pattern": [{"LOWER": {"IN": [
                "diabetes", "hypertension", "cancer", "asthma", "depression",
                "anxiety", "lupus", "epilepsy", "alzheimer", "parkinson",
                "stroke", "dementia", "arthritis", "copd", "fibromyalgia"
            ]}}]},
            {"label": "DISEASE", "pattern": [{"LOWER": "heart"}, {"LOWER":
{"IN": ["attack", "failure", "disease"]}}]},
            {"label": "DISEASE", "pattern": [{"LOWER": "myocardial"}, {"LOWER":
"infarction"}]},
            {"label": "DISEASE", "pattern": [{"LOWER": "multiple"}, {"LOWER":
"sclerosis"}]},
            {"label": "DISEASE", "pattern": [{"LOWER": "cystic"}, {"LOWER":
"fibrosis"}]},
            {"label": "DISEASE", "pattern": [{"LOWER": "sickle"}, {"LOWER":
"cell"}]},
            # Occupations
            {"label": "OCCUPATION", "pattern": [{"LOWER": {"IN": [
                "doctor", "physician", "nurse", "surgeon", "lawyer", "attorney",
                "teacher", "professor", "engineer", "developer", "programmer",
                "firefighter", "pilot", "chef", "accountant", "dentist",
                "pharmacist", "therapist", "scientist", "researcher", "manager",
                "consultant", "analyst", "mechanic", "electrician", "plumber",
                "cardiologist", "neurologist", "pediatrician", "psychiatrist"
            ]}}]},
            {"label": "OCCUPATION", "pattern": [{"LOWER": "police"}, {"LOWER":
"officer"}]},

```

```

        {"label": "OCCUPATION", "pattern": [{"LOWER": "software"}, {"LOWER":
{"IN": ["developer", "engineer"]}]}],
        {"label": "OCCUPATION", "pattern": [{"LOWER": "data"}, {"LOWER":
"scientist"}]}],

        # Age patterns
        {"label": "AGE", "pattern": [{"LIKE_NUM": True}, {"LOWER": {"IN":
["year", "years"]}]}], {"LOWER": "old"}]}],
        {"label": "AGE", "pattern": [{"LIKE_NUM": True}, {"TEXT": {"REGEX":
r"y\.?o\.?"}]}],
        {"label": "AGE", "pattern": [{"LOWER": "age"}, {"IS_PUNCT": True,
"OP": "?"}, {"LIKE_NUM": True}]}],

        # Gender patterns
        {"label": "GENDER", "pattern": [{"LOWER": {"IN": ["male", "female",
"man", "woman", "boy", "girl"]}]}],
    ]

    ruler.add_patterns(patterns)

# Age patterns - explicit age mentions (regex fallback)
AGE_PATTERNS = [
    r'\b(\d{1,3})\s*[-â€”]\s*(?:years\s*old|year\s*old|yo|y\.o\.|y/o)\b',
    r'\bage[d]?s*[:;]\s*(\d{1,3})\b',
    r'\b(\d{1,3})\s*(?:year|yr)[\s-]*old\b',
    r'\b([1-9][0-9]?)\s*[MFmf]\b',
]

# Date of birth patterns
DOB_PATTERNS = [
    r'\b(?:DOB|D\.O\.B\.|Date of Birth|Birth\s*Date|Born)[:\s]*(\d{1,2}[/-]
\d{1,2}[/-]\d{2,4})\b',
    r'\b(?:DOB|D\.O\.B\.|Date of Birth|Birth\s*Date|Born)[:\s]*(\d{4}[/-]
\d{1,2}[/-]\d{1,2})\b',
    r'\b(\d{1,2}[/-]\d{1,2}[/-]\d{4})\s*(?:\((DOB\)|\((Date of Birth\)))',
]

GENDER_MAPPING = {
    'male': 'male', 'm': 'male', 'man': 'male', 'boy': 'male',
    'female': 'female', 'f': 'female', 'woman': 'female', 'girl': 'female'
}

US_STATES = {
    'alabama', 'alaska', 'arizona', 'arkansas', 'california', 'colorado',
    'connecticut', 'delaware', 'florida', 'georgia', 'hawaii', 'idaho',
    'illinois', 'indiana', 'iowa', 'kansas', 'kentucky', 'louisiana',
    'maine', 'maryland', 'massachusetts', 'michigan', 'minnesota',
    'mississippi', 'missouri', 'montana', 'nebraska', 'nevada',
    'new hampshire', 'new jersey', 'new mexico', 'new york',
    'north carolina', 'north dakota', 'ohio', 'oklahoma', 'oregon',
    'pennsylvania', 'rhode island', 'south carolina', 'south dakota',
    'tennessee', 'texas', 'utah', 'vermont', 'virginia', 'washington',
    'west virginia', 'wisconsin', 'wyoming', 'district of columbia'
}

STATE_ABBREV_TO_FULL = {
    'al': 'alabama', 'ak': 'alaska', 'az': 'arizona', 'ar': 'arkansas',
    'ca': 'california', 'co': 'colorado', 'ct': 'connecticut', 'de':
'delaware',
    'fl': 'florida', 'ga': 'georgia', 'hi': 'hawaii', 'id': 'idaho',
    'il': 'illinois', 'in': 'indiana', 'ia': 'iowa', 'ks': 'kansas',
    'ky': 'kentucky', 'la': 'louisiana', 'me': 'maine', 'md': 'maryland',
    'ma': 'massachusetts', 'mi': 'michigan', 'mn': 'minnesota', 'ms':
'mississippi',

```

```

        'mo': 'missouri', 'mt': 'montana', 'ne': 'nebraska', 'nv': 'nevada',
        'nh': 'new hampshire', 'nj': 'new jersey', 'nm': 'new mexico', 'ny':
'new york',
        'nc': 'north carolina', 'nd': 'north dakota', 'oh': 'ohio', 'ok':
'oklahoma',
        'or': 'oregon', 'pa': 'pennsylvania', 'ri': 'rhode island', 'sc': 'south
carolina',
        'sd': 'south dakota', 'tn': 'tennessee', 'tx': 'texas', 'ut': 'utah',
        'vt': 'vermont', 'va': 'virginia', 'wa': 'washington', 'wv': 'west
virginia',
        'wi': 'wisconsin', 'wy': 'wyoming', 'dc': 'district of columbia'
    }
}

```

```

def extract(self, text: str) -> List[QuasiIdentifier]:
    """Extract quasi-identifiers using NLP + pattern matching."""
    qis = []

    # Process with spaCy
    doc = self.nlp(text)

    # Extract from NER
    qis.extend(self._extract_from_ner(doc, text))

    # Pattern-based extraction (fallback and supplement)
    qis.extend(self._extract_ages_pattern(text))
    qis.extend(self._extract_dobs(text))
    qis.extend(self._extract_zip_codes(text))
    qis.extend(self._extract_locations_from_ner(doc, text))

    # Deduplicate, preferring NER detections
    return self._deduplicate(qis)

def _extract_from_ner(self, doc, text: str) -> List[QuasiIdentifier]:
    """Extract QIs from spaCy NER results."""
    qis = []

    for ent in doc.ents:
        qi = None

        if ent.label_ == "DISEASE":
            qi = QuasiIdentifier(
                qi_type=QIType.DISEASE,
                raw_value=ent.text,
                normalized_value=ent.text.lower(),
                confidence=0.85,
                start_pos=ent.start_char,
                end_pos=ent.end_char,
                detection_method="ner"
            )
        elif ent.label_ == "OCCUPATION":
            qi = QuasiIdentifier(
                qi_type=QIType.OCCUPATION,
                raw_value=ent.text,
                normalized_value=ent.text.lower(),
                confidence=0.85,
                start_pos=ent.start_char,
                end_pos=ent.end_char,
                detection_method="ner"
            )
        elif ent.label_ == "AGE":
            # Extract numeric age
            age_match = re.search(r'\d+', ent.text)
            if age_match:
                age = int(age_match.group())

```

```

        if 0 <= age <= 120:
            qi = QuasiIdentifier(
                qi_type=QIType.AGE,
                raw_value=ent.text,
                normalized_value=str(age),
                confidence=0.9,
                start_pos=ent.start_char,
                end_pos=ent.end_char,
                detection_method="ner"
            )
        elif ent.label_ == "GENDER":
            gender_lower = ent.text.lower()
            if gender_lower in self.GENDER_MAPPING:
                qi = QuasiIdentifier(
                    qi_type=QIType.GENDER,
                    raw_value=ent.text,
                    normalized_value=self.GENDER_MAPPING[gender_lower],
                    confidence=0.95,
                    start_pos=ent.start_char,
                    end_pos=ent.end_char,
                    detection_method="ner"
                )
            elif ent.label_ == "GPE": # Geopolitical entity
                loc_lower = ent.text.lower()
                if loc_lower in self.US_STATES or loc_lower in
self.STATE_ABBREV_TO_FULL:
                    normalized = self.STATE_ABBREV_TO_FULL.get(loc_lower,
loc_lower)

                    qi = QuasiIdentifier(
                        qi_type=QIType.LOCATION,
                        raw_value=ent.text,
                        normalized_value=normalized,
                        confidence=0.85,
                        start_pos=ent.start_char,
                        end_pos=ent.end_char,
                        detection_method="ner"
                    )
            elif ent.label_ == "NORP": # Nationalities, religious, political
groups
                qi = QuasiIdentifier(
                    qi_type=QIType.ETHNICITY,
                    raw_value=ent.text,
                    normalized_value=ent.text.lower(),
                    confidence=0.7,
                    start_pos=ent.start_char,
                    end_pos=ent.end_char,
                    detection_method="ner"
                )

        if qi:
            qis.append(qi)

    return qis

def _extract_locations_from_ner(self, doc, text: str) ->
List[QuasiIdentifier]:
    """Extract location QIs from spaCy GPE entities."""
    qis = []

    for ent in doc.ents:
        if ent.label_ in ["GPE", "LOC"]:
            loc_lower = ent.text.lower()
            # Check if it's a US state
            if loc_lower in self.US_STATES:

```

```

        qis.append(QuasiIdentifier(
            qi_type=QIType.LOCATION,
            raw_value=ent.text,
            normalized_value=loc_lower,
            confidence=0.85,
            start_pos=ent.start_char,
            end_pos=ent.end_char,
            detection_method="ner"
        ))
    elif loc_lower in self.STATE_ABBREV_TO_FULL:
        qis.append(QuasiIdentifier(
            qi_type=QIType.LOCATION,
            raw_value=ent.text,
            normalized_value=self.STATE_ABBREV_TO_FULL[loc_lower],
            confidence=0.8,
            start_pos=ent.start_char,
            end_pos=ent.end_char,
            detection_method="ner"
        ))
    else:
        # Other locations (cities, etc.)
        qis.append(QuasiIdentifier(
            qi_type=QIType.LOCATION,
            raw_value=ent.text,
            normalized_value=ent.text.lower(),
            confidence=0.7,
            start_pos=ent.start_char,
            end_pos=ent.end_char,
            detection_method="ner"
        ))

    return qis

def _extract_ages_pattern(self, text: str) -> List[QuasiIdentifier]:
    """Extract ages using regex patterns."""
    qis = []
    for pattern in self.AGE_PATTERNS:
        for match in re.finditer(pattern, text, re.IGNORECASE):
            try:
                age = int(match.group(1))
                if 0 <= age <= 120:
                    qis.append(QuasiIdentifier(
                        qi_type=QIType.AGE,
                        raw_value=match.group(0),
                        normalized_value=str(age),
                        confidence=0.9,
                        start_pos=match.start(),
                        end_pos=match.end(),
                        detection_method="pattern"
                    ))
            except ValueError:
                continue
    return qis

def _extract_dobs(self, text: str) -> List[QuasiIdentifier]:
    """Extract dates of birth using regex patterns."""
    qis = []
    for pattern in self.DOB_PATTERNS:
        for match in re.finditer(pattern, text, re.IGNORECASE):
            dob_str = match.group(1)
            qis.append(QuasiIdentifier(
                qi_type=QIType.DATE_OF_BIRTH,
                raw_value=match.group(0),
                normalized_value=dob_str,

```

```

        confidence=0.95,
        start_pos=match.start(),
        end_pos=match.end(),
        detection_method="pattern"
    ))
    return qis

def _extract_zip_codes(self, text: str) -> List[QuasiIdentifier]:
    """Extract ZIP codes using regex patterns."""
    qis = []

    # ZIP with context
    context_pattern = r'(?::zip|postal|address|located|lives|resides)[^0-9]{0,30}(\d{5})(?:-\d{4})?'
    for match in re.finditer(context_pattern, text, re.IGNORECASE):
        qis.append(QuasiIdentifier(
            qi_type=QIType.ZIP_CODE,
            raw_value=match.group(1),
            normalized_value=match.group(1),
            confidence=0.95,
            start_pos=match.start(1),
            end_pos=match.end(1),
            detection_method="pattern"
        ))

    # State abbreviation + ZIP
    state_zip_pattern = r'\b[A-Z]{2}\s+(\d{5})(?:-\d{4})?\b'
    for match in re.finditer(state_zip_pattern, text):
        zip_code = match.group(1)
        if not any(qi.normalized_value == zip_code for qi in qis):
            qis.append(QuasiIdentifier(
                qi_type=QIType.ZIP_CODE,
                raw_value=match.group(0),
                normalized_value=zip_code,
                confidence=0.9,
                start_pos=match.start(1),
                end_pos=match.end(1),
                detection_method="pattern"
            ))

    return qis

def _deduplicate(self, qis: List[QuasiIdentifier]) -> List[QuasiIdentifier]:
    """Deduplicate QIs, preferring NER detections over pattern matches."""
    seen = {} # key -> (qi, priority)

    for qi in qis:
        key = (qi.qi_type, qi.normalized_value)
        priority = 1 if qi.detection_method == "ner" else 0

        if key not in seen or priority > seen[key][1]:
            seen[key] = (qi, priority)

    return [qi for qi, _ in seen.values()]

# =====
# Population Data Sources (same as before, abbreviated for space)
# =====
class PopulationDataSource(ABC):
    @abstractmethod
    def get_frequency(self, qi_type: QIType, value: str) ->
Optional[QIFrequency]:
    pass

```

```

class CensusDataSource(PopulationDataSource):
    """Census data source - implementation same as original."""
    BASE_URL = "https://api.census.gov/data"
    US_POPULATION = 331_900_000

    # ... (keep all the original implementation)
    # For brevity, I'm noting this should include all original methods

    def __init__(self, api_key: Optional[str] = None):
        self.api_key = api_key or os.getenv("CENSUS_API_KEY")
        self._age_distribution_cache: Optional[dict] = None

    def get_frequency(self, qi_type: QIType, value: str) ->
Optional[QIFrequency]:
        # Simplified - include full implementation from original
        if qi_type == QIType.AGE:
            return self._get_age_frequency(value)
        elif qi_type == QIType.GENDER:
            return self._get_gender_frequency(value)
        elif qi_type == QIType.LOCATION:
            return self._get_location_frequency(value)
        elif qi_type == QIType.ZIP_CODE:
            return self._get_zip_frequency(value)
        return None

    def _get_age_frequency(self, age_str: str) -> Optional[QIFrequency]:
        try:
            age = int(age_str)
        except ValueError:
            return None
        if age < 0 or age > 120:
            return None
        # Simplified fallback distribution
        prob = 0.01 # ~1% per year as rough estimate
        return QIFrequency(
            qi_type=QIType.AGE,
            value=str(age),
            probability=prob,
            population_count=int(prob * self.US_POPULATION),
            source="Census estimate"
        )

    def _get_gender_frequency(self, gender: str) -> Optional[QIFrequency]:
        gender_probs = {
            "female": 0.508, "male": 0.492,
        }
        prob = gender_probs.get(gender.lower(), 0.5)
        return QIFrequency(
            qi_type=QIType.GENDER,
            value=gender,
            probability=prob,
            population_count=int(prob * self.US_POPULATION),
            source="Census Bureau"
        )

    def _get_location_frequency(self, location: str) -> Optional[QIFrequency]:
        # Default for unknown locations
        return QIFrequency(
            qi_type=QIType.LOCATION,
            value=location,
            probability=0.02,
            population_count=int(0.02 * self.US_POPULATION),
            source="Default estimate"

```



```

    )

def _get_zip_frequency(self, zip_code: str) -> Optional[QIFrequency]:
    return QIFrequency(
        qi_type=QIType.ZIP_CODE,
        value=zip_code,
        probability=7500 / self.US_POPULATION,
        population_count=7500,
        source="Default ZIP estimate"
    )

class DiseaseDataSource(PopulationDataSource):
    """Disease prevalence data."""
    US_POPULATION = 331_900_000

    DISEASE_PREVALENCE = {
        "diabetes": 1/10, "hypertension": 1/3, "cancer": 1/200,
        "asthma": 1/13, "depression": 1/15, "anxiety": 1/5,
        "heart attack": 1/100, "myocardial infarction": 1/100,
        "stroke": 1/40, "alzheimer": 1/50, "dementia": 1/30,
        "multiple sclerosis": 1/1000, "lupus": 1/2000,
        "cystic fibrosis": 1/3500, "sickle cell": 1/365,
    }

    def get_frequency(self, qi_type: QIType, value: str) ->
Optional[QIFrequency]:
        if qi_type != QIType.DISEASE:
            return None

        disease_lower = value.lower().strip()
        for disease_key, prevalence in self.DISEASE_PREVALENCE.items():
            if disease_key in disease_lower or disease_lower in disease_key:
                return QIFrequency(
                    qi_type=QIType.DISEASE,
                    value=value,
                    probability=prevalence,
                    population_count=int(prevalence * self.US_POPULATION),
                    source="CDC prevalence data"
                )

        return QIFrequency(
            qi_type=QIType.DISEASE,
            value=value,
            probability=1/1000000,
            population_count=int(self.US_POPULATION / 1000000),
            source="Default rare disease estimate"
        )

class OccupationDataSource(PopulationDataSource):
    """Occupation data from BLS."""
    TOTAL_EMPLOYED = 160_000_000

    OCCUPATION_EMPLOYMENT = {
        "physician": 727000, "doctor": 727000, "nurse": 3100000,
        "teacher": 4500000, "lawyer": 813000, "engineer": 2000000,
        "software developer": 1850000, "cardiologist": 25000,
    }

    def get_frequency(self, qi_type: QIType, value: str) ->
Optional[QIFrequency]:
        if qi_type != QIType.OCCUPATION:
            return None

```

```

occupation_lower = value.lower().strip()
for occ_key, employment in self.OCCUPATION_EMPLOYMENT.items():
    if occ_key in occupation_lower or occupation_lower in occ_key:
        prob = employment / self.TOTAL_EMPLOYED
        return QIFrequency(
            qi_type=QIType.OCCUPATION,
            value=value,
            probability=prob,
            population_count=employment,
            source="BLS Statistics"
        )

return QIFrequency(
    qi_type=QIType.OCCUPATION,
    value=value,
    probability=100000 / self.TOTAL_EMPLOYED,
    population_count=100000,
    source="Default occupation estimate"
)

```

```

# =====
# Q-Score Calculator with Masking
# =====
class QScoreCalculator:
    """Calculates Q-Score based on k-anonymity principles with masking
    support."""

```

```

    REFERENCE_POPULATION = 331_900_000

```

```

    def __init__(self, k_threshold: int = 5):
        self.census = CensusDataSource()
        self.disease = DiseaseDataSource()
        self.occupation = OccupationDataSource()
        self.extractor = NLPQIExtractor()
        self.k_threshold = k_threshold

```

```

    def _mask_text(self, text: str, qis: List[QuasiIdentifier]) -> str:
        """Replace detected QIs with mask tokens."""
        if not qis:
            return text

        # Sort by start position in reverse
        sorted_qis = sorted(qis, key=lambda q: q.start_pos, reverse=True)

        masked = text
        for qi in sorted_qis:
            mask_token = f"[{qi.qi_type.value.upper()}]"
            masked = masked[:qi.start_pos] + mask_token + masked[qi.end_pos:]

        return masked

```

```

    def calculate(self, text: str) -> QScoreResult:
        """Calculate Q-Score and return result with masked text."""
        detected_qis = self.extractor.extract(text)

        if not detected_qis:
            return QScoreResult(
                q_score=0.0,
                expected_k=self.REFERENCE_POPULATION,
                detected_qis=[],
                frequencies={},
                explanation="No quasi-identifiers detected.",
            )

```

```

        original_text=text,
        masked_text=text
    )

    # Look up frequencies
    frequencies = {}
    for qi in detected_qis:
        freq = self._get_frequency(qi)
        if freq:
            frequencies[qi] = freq

    if not frequencies:
        masked_text = self._mask_text(text, detected_qis)
        return QScoreResult(
            q_score=0.0,
            expected_k=self.REFERENCE_POPULATION,
            detected_qis=detected_qis,
            frequencies={},
            explanation=f"Detected {len(detected_qis)} QI(s) but could not
lookup frequencies.",
            original_text=text,
            masked_text=masked_text
        )

    # Calculate joint probability
    joint_prob = 1.0
    for freq in frequencies.values():
        joint_prob *= freq.probability

    # Expected equivalence class size
    expected_k = self.REFERENCE_POPULATION * joint_prob

    # Convert to Q-Score
    if expected_k >= self.k_threshold:
        q_score = 0.0
    else:
        q_score = min(1.0, 1.0 - (expected_k / self.k_threshold))

    # Generate explanation
    explanation = self._generate_explanation(detected_qis, frequencies,
expected_k)

    # Generate masked text
    masked_text = self._mask_text(text, detected_qis)

    return QScoreResult(
        q_score=q_score,
        expected_k=expected_k,
        detected_qis=detected_qis,
        frequencies=frequencies,
        explanation=explanation,
        original_text=text,
        masked_text=masked_text
    )

def _get_frequency(self, qi: QuasiIdentifier) -> Optional[QIFrequency]:
    if qi.qi_type in [QIType.AGE, QIType.DATE_OF_BIRTH, QIType.GENDER,
        QIType.LOCATION, QIType.ZIP_CODE]:
        return self.census.get_frequency(qi.qi_type, qi.normalized_value)
    elif qi.qi_type == QIType.DISEASE:
        return self.disease.get_frequency(qi.qi_type, qi.normalized_value)
    elif qi.qi_type == QIType.OCCUPATION:
        return self.occupation.get_frequency(qi.qi_type,
qi.normalized_value)

```

```

        return None

    def _generate_explanation(self, detected_qis: list, frequencies: dict,
expected_k: float) -> str:
        lines = [f"Detected {len(detected_qis)} quasi-identifier(s):"]

        for qi in detected_qis:
            freq = frequencies.get(qi)
            method = f"[{qi.detection_method}]"
            if freq:
                lines.append(
                    f"  - {qi.qi_type.value}: '{qi.normalized_value}' {method} "
                    f"(P ≈ {freq.probability:.2e}, ~{freq.population_count:,}
people)"
                )
            else:
                lines.append(
                    f"  - {qi.qi_type.value}: '{qi.normalized_value}' {method}
(frequency unknown)"
                )

            lines.append(f"\nExpected equivalence class size E[k]:
{expected_k:.4f}")

            if expected_k < 1:
                lines.append("ðŸ§Œ E[k] < 1: Likely UNIQUELY IDENTIFYING")
            elif expected_k < self.k_threshold:
                lines.append(f"âŸ§ ï, E[k] < {self.k_threshold}: HIGH re-
identification risk")
            else:
                lines.append(f"âœ“ E[k] â‰¥ {self.k_threshold}: Acceptable k-
anonymity")

        return "\n".join(lines)

# =====
# Main Interface
# =====
class QScoreAnalyzer:
    """Main interface for Q-Score analysis."""

    def __init__(self, k_threshold: int = 5):
        self.calculator = QScoreCalculator(k_threshold=k_threshold)

    def analyze(self, text: str) -> QScoreResult:
        return self.calculator.calculate(text)

    def get_score(self, text: str) -> float:
        return self.calculator.calculate(text).q_score

    def get_report(self, text: str) -> dict:
        result = self.calculator.calculate(text)

        return {
            "q_score": round(result.q_score, 4),
            "expected_k": round(result.expected_k, 4),
            "risk_level": "HIGH" if result.q_score >= 0.7 else "MEDIUM" if
result.q_score >= 0.3 else "LOW",
            "k_threshold": self.calculator.k_threshold,
            "detected_quasi_identifiers": [
                {
                    "type": qi.qi_type.value,
                    "raw_value": qi.raw_value,

```

```

        "normalized_value": qi.normalized_value,
        "confidence": qi.confidence,
        "detection_method": qi.detection_method,
        "position": {"start": qi.start_pos, "end": qi.end_pos},
        "frequency": {
            "probability": result.frequencies[qi].probability,
            "population_count":
result.frequencies[qi].population_count,
            "source": result.frequencies[qi].source
        } if qi in result.frequencies else None
    }
    for qi in result.detected_qis
],
"explanation": result.explanation,
"masked_text": result.masked_text
}

```

```

# =====
# Public API
# =====
def calculate_qscore(text: str) -> dict:
    """Calculate Q-Score for text with masking."""
    analyzer = QScoreAnalyzer()
    return analyzer.get_report(text)

if __name__ == "__main__":
    sample = """
    Patient [PERSON] was seen at Boston Medical Center.
    Dr. [PERSON] prescribed medication for hypertension.
    Contact: [URL]ADDRESS]HONE_NUMBER]. [ORGANIZATION]: [US_SSN].
    """

    report = calculate_qscore(sample)
    print(json.dumps(report, indent=2))

```